FINAL_REPORT – SPRINT 1[Ankit Kumar]

Project Title: ProfileBook – A Social Networking Platform

Student Name: [Ankit Kumar]

Date: March 16, 2026

Batch: [WIPRO NGA - .Net Full Stack Angular - FY26 - C4 tech] | **Instructor:** [Ramesh Nediyadadh]

Table of Contents

1. Problem Definition and Objectives
2. Frontend & Backend Architecture
3. Component Breakdown & API Design
4. Database – Schema, Tables & Relationships
5. User Registration & Login Operations
6. Admin Login & CRUD Operations
7. Angular Front-End Template
8. Code Sanitization – Structure & Components
9. CRUD Functionalities (Admin & User)
10. Advanced Feature – Post Moderation Workflow
11. Responsiveness & Client-Side Validation
12. Session Handling
13. Angular–Backend Integration & JWT Security
14. Initial Unit Testing

1. Problem Definition and Objectives

1.1 Problem Statement

Social platforms today either lack robust administrative oversight or prioritize complexity over usability. **ProfileBook** addresses this by providing:

- A **clean, secure entry point** for both users and administrators.
- An **admin-first content moderation** model where posts require approval before publishing.
- A **unified user model** with role-based access — no separate admin database tables.

1.2 Objectives

Criteria	Goal
Database	Core schema for Users, Posts, Groups with EF Core relationships
Admin Login	Admin auth via same login endpoint + role-based dashboard access
User Registration & Login	Register/Login with HMACSHA512 hashing + JWT + validation
Angular Template	Routing, Header/Sidebar, Login page, Admin & User dashboard shell
Code Sanitization	Modular components, service pattern, clean folder structure
CRUD Functionalities	User registration, post creation, admin user list + post approval
Advanced Feature	Post Approval Workflow (Pending → Approved)
Responsiveness	CSS Flexbox/Grid responsive pages + form validation
Session Handling	JWT in localStorage + AuthGuard + AdminGuard
JWT Integration	Bearer token sent with every secured API call

Testing	auth.spec.ts, user.spec.ts, post.spec.ts with Jasmine/Karma

2. Frontend & Backend Architecture

2.1 Technology Stack

Layer	Technology	Purpose
Frontend	Angular 19 (Standalone Components), TypeScript, RxJS	Single Page Application
Backend	ASP.NET Core 8 Web API	RESTful API services
Database	SQL Server + Entity Framework Core 8	Relational data storage
Security	JWT Bearer (HMACSHA512)	Stateless auth with roles
Dev Tools	VS Code, Git, GitHub	Source control & development

2.2 High-Level System Flow

```
[Angular SPA] ──POST /api/auth/login──▶ [ASP.NET Core API]
◀── JWT Token ──────────── [AuthController] │
──GET /api/user (Bearer)──▶ [UserController]
◀── JSON Response ──────── [→ EF Core → SQL Server]
```

1. User/Admin fills login form → Angular sends credentials backend.
2. API validates credentials, returns signed JWT with `role` claim.
3. Angular stores JWT in `localStorage`; guards check it on every route navigation.
4. Subsequent API calls attach the token as `Authorization: Bearer <token>`.

3. Component Breakdown & API Design

3.1 Frontend Components

Area	Component / File	Role
Auth	`auth/login/login.ts`	Login form + role-based navigation
Auth	`auth/register/register.ts`	User self-registration
Shared	`shared/navbar/navbar.ts`	Global top navigation bar
Shared	`shared/sidebar/sidebar.ts`	Role-aware side navigation
User	`pages/user-dashboard/user-dashboard.ts`	User layout shell
Admin	`pages/admin-dashboard/admin-layout.ts`	Admin layout shell
Admin	`admin-dashboard/stats/admin-dashboard.ts`	Statistics/overview page
Admin	`admin-dashboard/users/user-management.ts`	User CRUD panel
Admin	`admin-dashboard/approvals/post-approvals.ts`	Post moderation panel
Guards	`core/guards/auth.guard.ts`	AuthGuard + AdminGuard

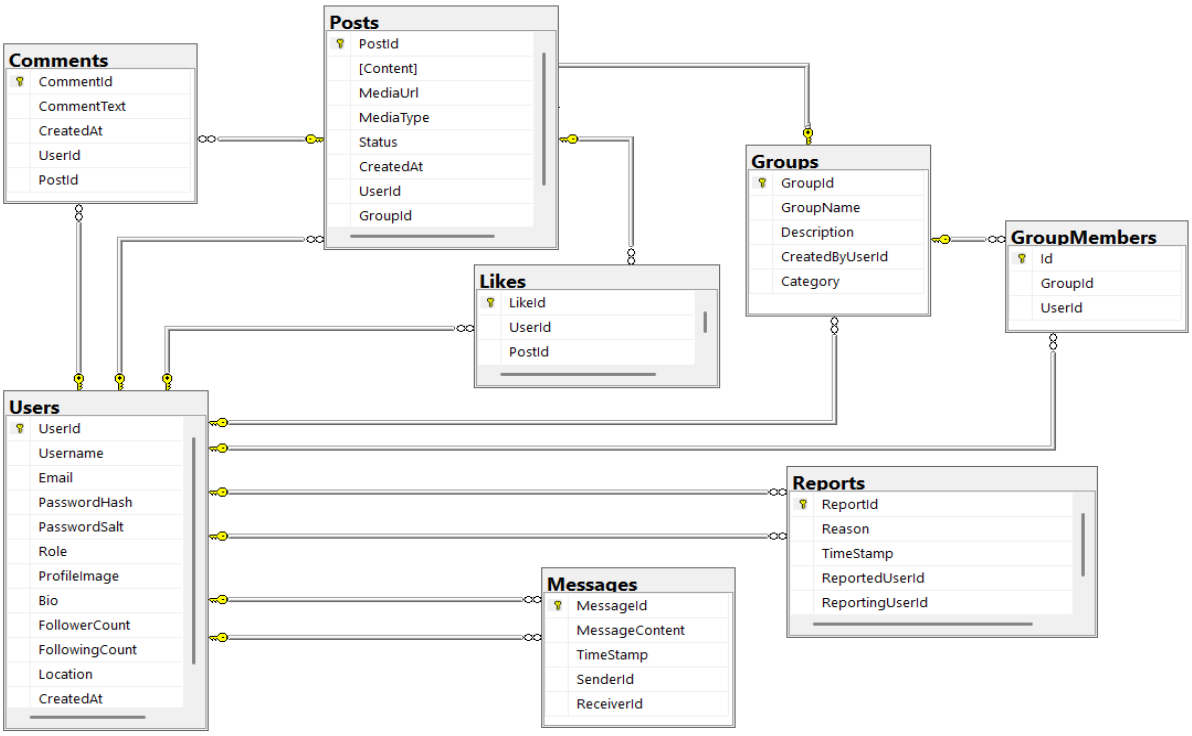
3.2 Core API Endpoints

Method	Endpoint	Auth Required	Purpose
POST	/api/auth/register	Public	Create new user
POST	/api/auth/login	Public	Issue JWT token
GET	/api/auth/profile	[Authorize]	Get own profile
GET	/api/user	Admin only	List all users
POST	/api/user	Admin only	Create a user
DELETE	/api/user/{id}	Admin only	Delete a user

GET	/api/post/pending	Admin only	Get posts awaiting approval
PUT	/api/post/{id}/approve	Admin only	Approve a post
PUT	/api/post/{id}/reject	Admin only	Reject a post

4. Database – Schema, Tables & Relationships

4.1 Entity Relationship Summary



USERS —< POSTS (One user creates many posts)
USERS —< COMMENTS (One user writes many comments)
USERS —< REPORTS (One user can report / be reported)

4.2 Users Table

Column	Type	Constraints	Description

UserId	int	PK, Identity	Auto-incremented unique ID
Username	string	Required, MaxLength(50)	Login handle
Email	string	Required, [EmailAddress]	Contact address
PasswordHash	byte[]	Required	HMACSHA512 hash
PasswordSalt	byte[]	Required	Random salt for hashing
Role	string	Default: "User"	"User" or "Admin"
Bio	string?	MaxLength(500)	Optional profile bio
CreatedAt	DateTime	Default: UtcNow	Registration timestamp

Example Data:

UserId	Username	Email	Role	CreatedAt
1	admin	admin@profilebook.com	**Admin**	2026-03-01
2	ankit	ankit@email.com	User	2026-03-15
3	nikhil	nikhil@email.com	User	2026-03-16

C# Model (Models/User.cs):

```
public class User
{
    [Key]
    public int UserId { get; set; }

    [Required, MaxLength(50)]
    public string Username { get; set; } = string.Empty;

    [Required, EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required] public byte[] PasswordHash { get; set; }
    [Required] public byte[] PasswordSalt { get; set; }

    [Required]
    public string Role { get; set; } = "User"; // "User" or "Admin"
```

```
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public List<Post> Posts { get; set; } = new();
}
```

4.3 Posts Table

Column	Type	Constraints	Description
PostId	int	PK, Identity	Auto-incremented unique ID
Content	string	Required, MaxLength(500)	Post body text
Status	string	Default: "Pending"	Approval workflow state
UserId	int	FK → Users	Owner of the post
CreatedAt	DateTime	Default: UtcNow	Post timestamp

Example Data:

PostId	Content	Status	UserId
101	Hello ProfileBook!	Approved	2
102	Exciting tech news...	Pending	3
103	Admin announcement	Approved	1

C# Model (Models/Post.cs):

```
public class Post
{
    [Key]
    public int PostId { get; set; }

    [Required, MaxLength(500)]
    public string Content { get; set; } = string.Empty;

    [Required]
    public string Status { get; set; } = "Pending"; // Pending/Approved/Rejected
```

```
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public int UserId { get; set; }
    public User? User { get; set; }
}
```

4.4 Groups Table (Basic Level)

Column	Type	Description
GroupId	int (PK)	Unique identifier
GroupName	string (Required)	Display name of the group
Category	string?	Tag for filtering
CreatedByUserId	int (FK → Users)	Creator reference

Example Data:

GroupId	GroupName	Category	CreatedByUserId
1	Developers Hub	Tech	1
2	Photography Club	Arts	2

4.5 DataContext – Relationship Configuration

```
// Data/DataContext.cs
public class DataContext : DbContext
{
    public DbSet<User>  Users  => Set<User>();
    public DbSet<Post>  Posts  => Set<Post>();
    public DbSet<Group> Groups => Set<Group>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
 // ── Post ──────────────────────────────────────────
        modelBuilder.Entity<Post>()
            .HasOne(p => p.User)
            .WithMany(u => u.Posts)
            .HasForeignKey(p => p.UserId)
```

```
            .OnDelete(DeleteBehavior.Cascade); // Posts deleted with user

        // ── Group ──────────────────────────────────────────
        modelBuilder.Entity<Group>()
            .HasOne(g => g.CreatedByUser)
            .WithMany()
            .HasForeignKey(g => g.CreatedByUserId)
            .OnDelete(DeleteBehavior.NoAction); // No cascade to avoid cycles
    }
}
```

Why Cascade vs NoAction? When a user is deleted, their posts should be cleaned automatically (Cascade). But deleting a user should NOT delete the group they created — the group should persist (NoAction).

5. User Registration & Login Operations

5.1 Backend – Registration Endpoint

Passwords are **never stored in plain text**. We use **HMACSHA512**, a cryptographic one-way hash algorithm. Each registration generates a random 128-byte **salt** (stored alongside the user), and a 64-byte **hash** derived from the password combined with the salt. This means even if two users share the same password, their stored hashes will be completely different — making database-level brute-force attacks practically impossible. The endpoint also checks **duplicate usernames** before creating the account, returning a descriptive error message if the username is already taken.

```
// AuthController.cs
[HttpPost("register")]
public async Task<IActionResult> Register(UserRegisterDto request)
{
    // Step 1: Reject duplicate usernames
    if (await _context.Users.AnyAsync(u => u.Username == request.Username))
        return BadRequest("User already exists.");
    // Step 2: Hash the password with a unique salt
    CreatePasswordHash(request.Password, out byte[] hash, out byte[] salt);

    // Step 3: Persist the new user with default "User" role
    var user = new User {
```

```
        Username     = request.Username,
        Email        = request.Email,
        PasswordHash = hash,
        PasswordSalt = salt,
        Role         = "User"
    };
    _context.Users.Add(user);
    await _context.SaveChangesAsync();
    return Ok(new UserResponseDto { UserId = user.UserId, Username = user.Username,
                                    Email  = user.Email,  Role     = user.Role });
}

private void CreatePasswordHash(string password, out byte[] hash, out byte[] salt)
{
    using var hmac = new HMACSHA512();
    salt = hmac.Key;                                          // 128-byte random
salt
    hash = hmac.ComputeHash(Encoding.UTF8.GetBytes(password)); // 64-byte hash
}
```

5.2 Backend – Login & JWT Issuance

The login endpoint performs two sequential security checks: it first verifies the username exists, then re-hashes the submitted password using the stored salt and compares it byte-by-byte to the stored hash. If both checks pass, `CreateToken(user)` is called, building a **signed JWT** containing three claims — the user's ID, username, and role. The token is signed using HMAC-SHA256 with a secret key from `appsettings.json` and expires after 24 hours, after which the user must log in again. The role claim embedded in the token is what ASP.NET uses server-side to enforce `[Authorize(Roles = "Admin")]`.

```
[HttpPost("login")]
public async Task<ActionResult> Login(UserLoginDto request)
{
    var user = await _context.Users
        .FirstOrDefaultAsync(u => u.Username == request.Username);

    if (user == null)
        return BadRequest("User not found.");

    if (!VerifyPasswordHash(request.Password, user.PasswordHash, user.PasswordSalt))
        return BadRequest("Wrong password.");

    string token = CreateToken(user);
```

```
    return Ok(new { token, userId = user.UserId, username = user.Username, role =
user.Role });
}

private string CreateToken(User user)
{
    var claims = new List<Claim> {
        new(ClaimTypes.NameIdentifier, user.UserId.ToString()),
        new(ClaimTypes.Name,           user.Username),
        new(ClaimTypes.Role,           user.Role)  // ← "User" or "Admin"
    };
    var key   = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["JwtSettings:Token"]!));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
    var token = new JwtSecurityToken(
        issuer: "ProfileBook", audience: "ProfileBookUsers",
        claims: claims, expires: DateTime.Now.AddHours(24), signingCredentials: creds
    );
    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

5.3 Frontend – Login Component

Before making any API call, the Angular component performs **client-side validation** — ensuring both fields contain non-whitespace content. This prevents unnecessary HTTP requests and gives the user immediate visual feedback. On a successful response, the `role` value returned by the API determines where the user is redirected: `Admin` accounts go to the admin panel, while regular `User` accounts are sent to the social feed. Error handling distinguishes between a wrong-credentials error (HTTP 400) and any other unexpected server errors, providing a user-friendly message in each case.

```
// auth/login/login.ts
onSubmit() {
  this.formSubmitted = true;
  this.errorMessage  = '';

  // Client-side validation: ensure fields are not empty
  if (!this.loginData.username.trim() || !this.loginData.password.trim()) {
    this.errorMessage = 'Please fill in all fields.';
    return;
  }

  this.isLoading = true;
  this.auth.login(this.loginData).subscribe({
    next: (res: any) => {
```

```
      this.isLoading = false;
      // Role-based redirect
      if (res.role === 'Admin') {
        this.router.navigate(['/admin-dashboard']);
      } else {
        this.router.navigate(['/dashboard/feed']);
      }
    },
    error: (err: any) => {
      this.isLoading = false;
      this.errorMessage = err.status === 400
        ? 'Incorrect username or password.'
        : 'An unexpected error occurred. Please try again.';
    }
  });
```

6. Admin Login & CRUD Operations

Admins use the **same `/api/auth/login` endpoint** as regular users. The `Role = "Admin"` field in the database is the sole differentiator.

6.1 Admin Account (Seeded)

Rather than exposing a separate admin registration endpoint, the admin account is **seeded directly into the database** at startup via a database initializer. This is a secure and common enterprise pattern — the admin credentials are configured once during deployment and are never accessible through any public API endpoint. The seeded account uses the same HMACSHA512 hashing as regular users, so the password is never stored in plain text, even in the seed data.

```
var adminUser = new User {
    Username     = "admin",
    Email        = "admin@profilebook.com",
    PasswordHash = hash,    // Hashed "Admin@123"
    PasswordSalt = salt,
    Role         = "Admin"
};
```

6.2 Role-Based Redirect After Login

```
// Frontend login.ts
if (res.role === 'Admin') {
  this.router.navigate(['/admin-dashboard']); // → Admin Panel
} else {
  this.router.navigate(['/dashboard/feed']);  // → User Feed
}
```

6.3 Secured Admin Controller

The `[Authorize(Roles = "Admin")]` attribute is applied at the **class level**, meaning every single method inside this controller is automatically protected — no need to add it to each endpoint individually. If a request arrives without a JWT, ASP.NET returns **401 Unauthorized**. If the token is valid but the role claim is not `"Admin"`, it returns **403 Forbidden**. This means even a fully authenticated regular user cannot call these endpoints — the backend enforces security independently of what the Angular frontend shows or hides.

```
[ApiController]
[Route("api/[controller]")]
[Authorize(Roles = "Admin")]   // ← All actions require Admin JWT
public class UserController : ControllerBase
{
    [HttpGet]                                           // READ: List all users
    public async Task<IActionResult> GetAllUsers()
        => Ok(await _userService.GetAllUsers());

    [HttpPost]                                          // CREATE: Add new user
    public async Task<IActionResult> CreateUser(UserCreateDto request)
    {
        var user = await _userService.CreateUser(request);
        return Ok(user);
    }

    [HttpDelete("{id}")]                                // DELETE: Remove a user
    public async Task<IActionResult> DeleteUser(int id)
    {
        await _userService.DeleteUser(id);
        return NoContent();
    }
}
```

7. Angular Front-End Template

7.1 Full Routing Configuration

Angular's router is configured with a **nested (child) route** structure. The user dashboard and admin dashboard act as **layout shells** — they render the sidebar and navbar once — and their child routes (Feed, Profile, Stats, Users) display inside a `<router-outlet>`. This avoids duplicating layout code across every page. The admin route chains **two guards**: `AuthGuard` first ensures the visitor is logged in, then `AdminGuard` ensures their stored role is `"Admin"`. Both must pass for the route to activate — if either fails, the user is redirected automatically.

```
// app-routing-module.ts
const routes: Routes = [
  { path: '',         redirectTo: 'login', pathMatch: 'full' },
  { path: 'login',    component: Login },
  { path: 'register', component: Register },

  // Protected User Shell
  { path: 'dashboard', component: UserDashboard, canActivate: [AuthGuard], children:
[
    { path: 'feed',    component: Feed },
    { path: 'profile', component: Profile },
  ]},

  // Protected Admin Shell (requires BOTH AuthGuard AND AdminGuard)
  { path: 'admin-dashboard', component: AdminLayoutComponent,
    canActivate: [AuthGuard, AdminGuard], children: [
      { path: '',           redirectTo: 'stats', pathMatch: 'full' },
      { path: 'stats',      component: AdminDashboardComponent },
      { path: 'users',      component: UserManagementComponent },
      { path: 'approvals',  component: PostApprovalsComponent },
  ]},
];
```

7.2 Login Page Layout

The login page uses a **split-screen design**:

- **Left side:** Minimalist branding (logo + tagline) with an indigo-to-violet gradient.
- **Right side:** Clean authentication form.
- **Footer:** Exclusive to the login page only — absent from user/admin panels.

ProfileBook

The modern way to connect and share.

Welcome Back

Please enter your details to sign in

Username

Enter your username

Password

••••••••

Sign In

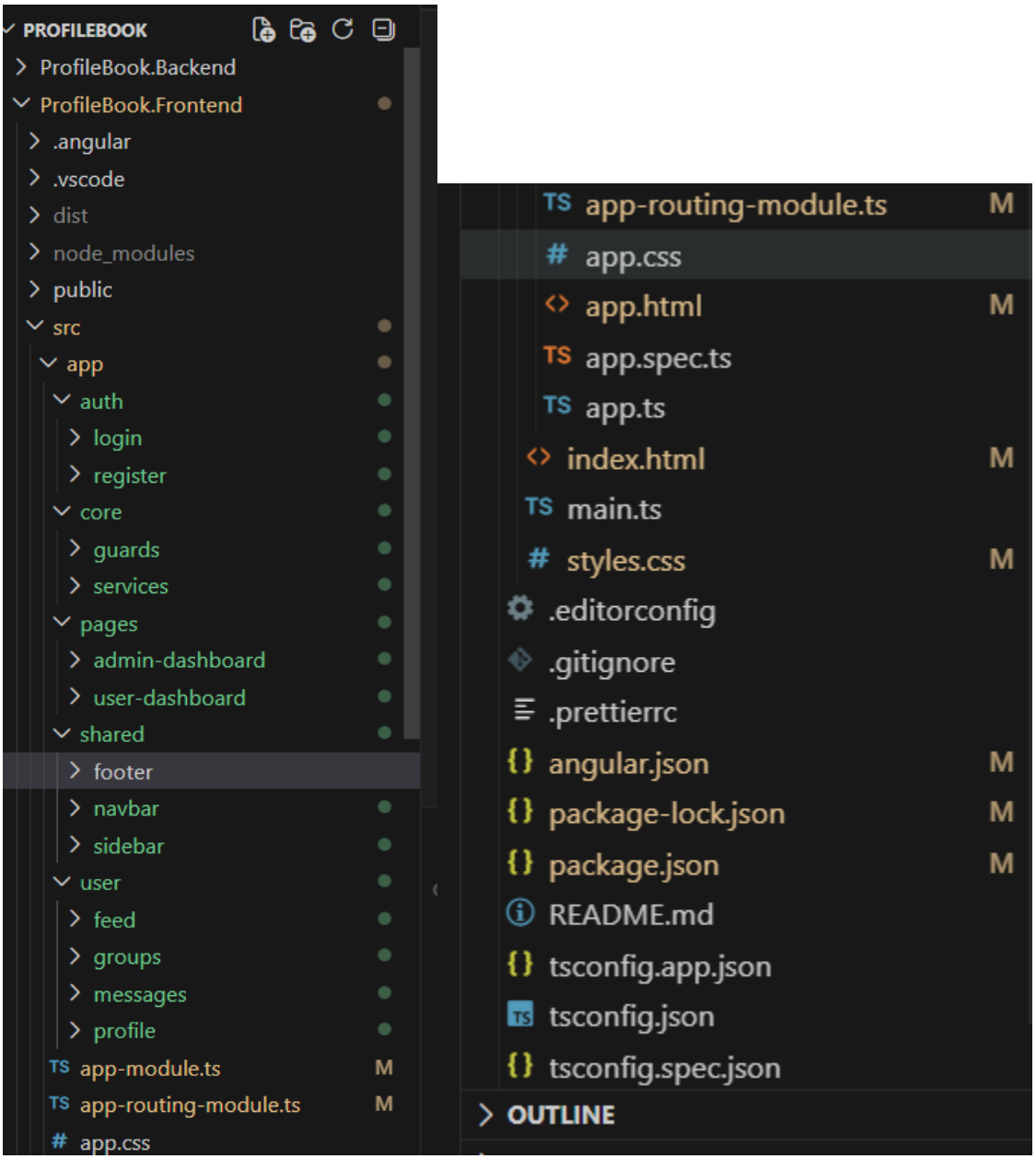
New here? Create an account

About Privacy Terms Contact

© 2024 ProfileBook. All rights reserved.

8. Code Sanitization – Structure & Components

8.1 Angular Folder Structure



8.2 Service Pattern – Separation of Concerns

In Angular, it is a best practice to keep **components thin** — they should only manage what the user sees, not how data is obtained. All HTTP logic lives inside **injectable services** in the

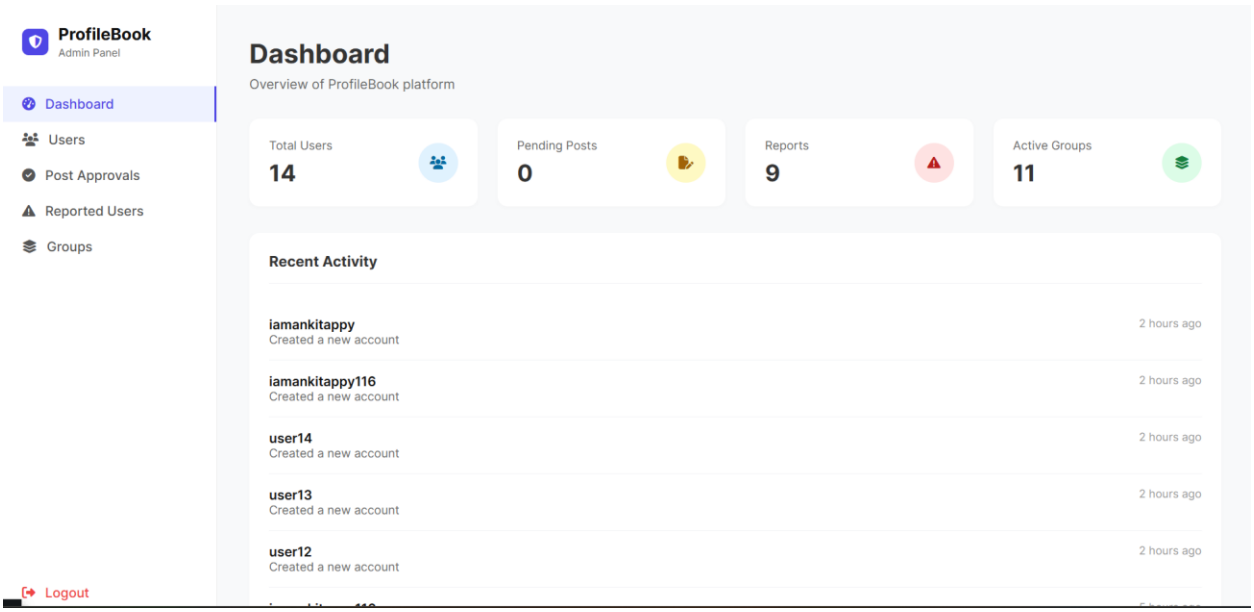
`core/services/` folder. The `Auth` service uses RxJS's `tap` operator to perform a **side-effect** (storing tokens in `localStorage`) inside the observable pipeline without modifying the data stream. Notice how four separate key-value pairs are stored: the **token** is needed for API calls, the **username** for UI display, the **userId** for ownership-based API requests, and the **role** for route guard checks — each serving a distinct purpose.

```
// core/services/auth.ts
@Injectable({ providedIn: 'root' })
export class Auth {
  private apiUrl = 'http://localhost:5134/api/auth';
  constructor(private http: HttpClient) {}

  // All HTTP logic lives in the service — components stay clean
  login(loginData: any): Observable<any> {
    return this.http.post<any>(`${this.apiUrl}/login`, loginData).pipe(
      tap((res: any) => {
        if (res?.token) {
          localStorage.setItem('token',    res.token);
          localStorage.setItem('username', res.username);
          localStorage.setItem('userId',   res.userId);
          localStorage.setItem('role',     res.role);
        }
      })
    );
  }

  isLoggedIn(): boolean {
    return !!localStorage.getItem('token');
  }
}
```

9. CRUD Functionalities (Admin & User)

Actor	Entity	Operation	Implementation
User	Account	CREATE	POST /api/auth/register
User	Account	READ	GET /api/auth/profile
User	Post	CREATE	POST /api/post (status=Pending)

User	Post	READ	GET /api/post (only Approved)
Admin	Users	READ	GET /api/user → UserManagementComponent
Admin	Users	CREATE	POST /api/user → modal form
Admin	Users	DELETE	DELETE /api/user/{id}
Admin	Posts	READ	GET /api/post/pending
Admin	Posts	UPDATE	PUT /api/post/{id}/approve or /reject

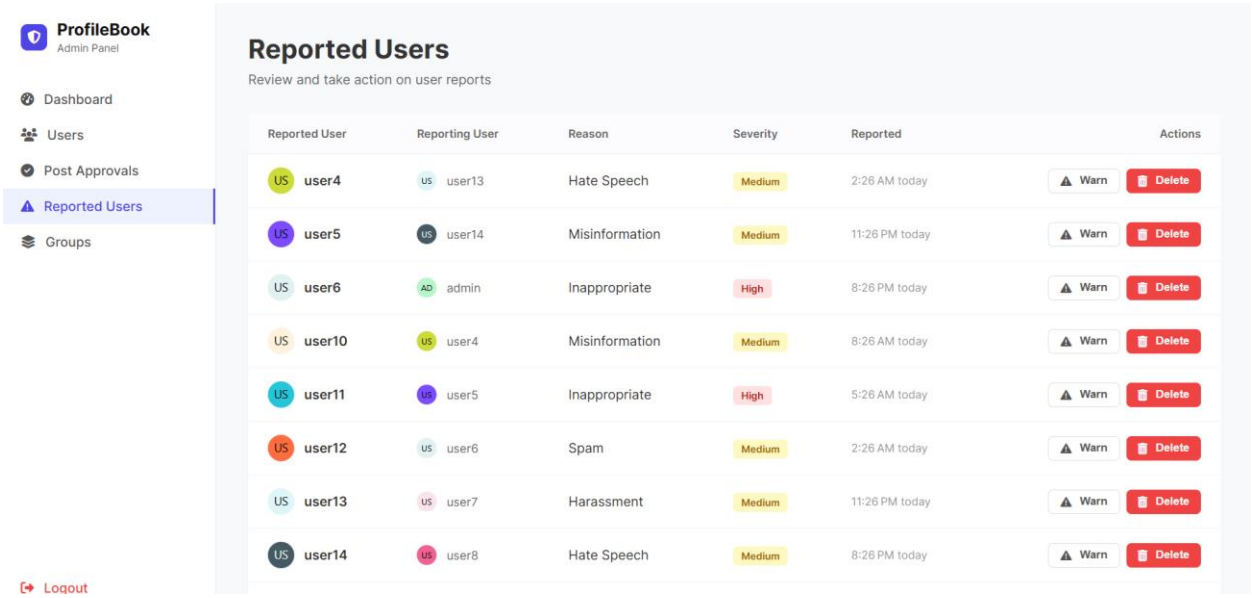


Admin User Management (Angular)

The `UserManagementComponent` fetches the user list as soon as it initializes using Angular's `ngOnInit` lifecycle hook. The `loadUsers()` method is extracted as a separate function so it can be called both on page load *and* immediately after a delete operation, keeping the displayed list always in sync with the server without requiring a full page refresh. The `deleteUser` method shows a native browser **confirmation dialog** before making the API call — a simple but important UX safeguard that prevents accidental permanent deletions. `//user-management.ts`

```
ngOnInit(): void {
  this.loadUsers();
}

loadUsers() {
  this.userService.getAllUsers().subscribe({
```

```
    next: (users: any[]) => this.users = users,
    error: (err) => console.error('Could not load users', err)
  });
}

deleteUser(id: number) {
  if (!confirm('Permanently delete this user?')) return;
  this.userService.deleteUser(id).subscribe(() => this.loadUsers());
}
```



10. Advanced Feature – Post Moderation Workflow 8 pts

Why This Is an Advanced Feature

A basic CRUD system would publish posts immediately. Our **Post Approval Workflow** implements a *content restriction pipeline*:

1. User creates a post → Status is automatically set to `"Pending"`.
2. Admin reviews it in the Approvals panel.
3. Admin approves or rejects → Status becomes `"Approved"` or `"Rejected"`.
4. Only `"Approved"` posts appear in the public feed.

Backend – Moderation Endpoints

Three dedicated endpoints handle the content moderation lifecycle. All three are secured with `[Authorize(Roles = "Admin")]`, making it impossible for a regular user to invoke them — even directly via API tools like Postman — without a valid Admin JWT. The `GetPendingPosts` endpoint gives the admin a queue of content to review. The `ApprovePost` and `RejectPost` endpoints then update the `Status` field on the post record in the database. This status change is what the feed query reads to decide which posts are visible to the public.

```
// Controllers/PostController.cs
[HttpGet("pending")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> GetPendingPosts() {
    var posts = await _postService.GetPendingPosts();
    return Ok(posts);
}

[HttpPut("{id}/approve")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> ApprovePost(int id) {
    var result = await _postService.ApprovePost(id);
    return Ok(result);
}

[HttpPut("{id}/reject")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> RejectPost(int id) {
    var result = await _postService.RejectPost(id);
    return Ok(result);
}
```

Feed Filtering – Only Approved Posts

The public feed service applies a strict **LINQ `.Where` filter** that queries only posts with a status of `"Approved"`. This means Pending and Rejected posts are structurally invisible to regular users — they are never included in any API response the user receives. Posts are also ordered by `CreatedAt` in descending order so the most recent approved content always appears at the top of the feed. This approach keeps the data layer clean — status filtering happens at the database query level, not in application memory.

```
// Services/PostService.cs
public async Task<List<PostResponseDto>> GetApprovedPosts() {
```

```
    return await _context.Posts
        .Where(p => p.Status == "Approved")   // ← Strict filter
        .OrderByDescending(p => p.CreatedAt)
        .Select(p => new PostResponseDto { ... })
        .ToListAsync();
}
```

11. Responsiveness & Client-Side Validation

11.1 CSS Responsive Grid

The admin dashboard uses CSS **Grid with `auto-fit` and `minmax()`** — a powerful combination that automatically determines how many columns to render based on the available screen width, with no JavaScript involved. Each stat card will be at least 240px wide; on narrower screens, cards automatically wrap to the next row. The login page uses a **media query** at the 768px tablet breakpoint to hide the decorative branding panel, keeping the mobile experience clean and entirely focused on the login form itself.

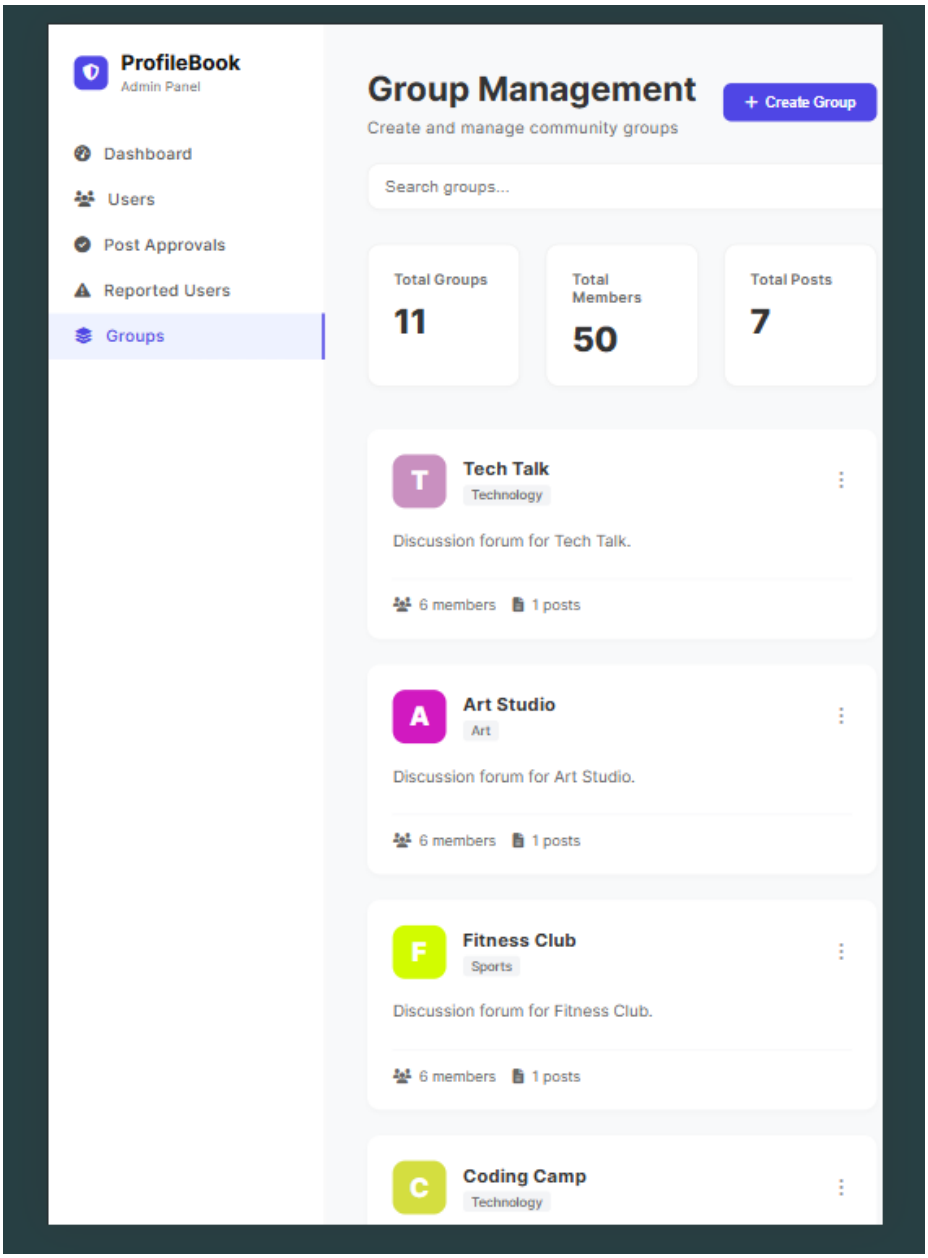
```
/* admin-dashboard.css */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 24px;
  /* Mobile: 1 column | Tablet: 2 columns | Desktop: 4 columns */
}

/* login.css */
@media (max-width: 768px) {
  .brand-side { display: none; }    /* Hide branding on small screens */
  .form-side  { width: 100%; }      /* Full-width login on mobile */
}
```

11.2 Client-Side Form Validation

Client-side validation serves two important purposes: it provides **immediate visual feedback** to the user without waiting for a network round-trip, and it reduces unnecessary load on the backend server. The guard clause checks both fields using `.trim()` to prevent whitespace-only submissions — a common edge case. If validation fails, the function returns early and the HTTP

request is **never made**. The Angular HTML template then conditionally displays the error message using `*ngIf`, ensuring it only appears when there is actually an error to show.

```
// login.ts
onSubmit() {
  if (!this.loginData.username.trim() || !this.loginData.password.trim()) {
    this.errorMessage = 'Please fill in all fields.';
    return;  // ← API is never called if validation fails
  }
  // ...proceed with HTTP call
}
<!-- login.html – error message shown only when set -->
<div class="error-msg" *ngIf="errorMessage">
  <i class="fas fa-exclamation-circle"></i> {{ errorMessage }}
</div>
```



12. Session Handling

12.1 Token Storage After Login

```
// core/services/auth.ts
tap((res: any) => {
  if (res?.token) {
    localStorage.setItem('token',    res.token);      // JWT Bearer token
```

```
    localStorage.setItem('username', res.username);   // Display in navbar
    localStorage.setItem('userId',   res.userId);     // For API ownership
    localStorage.setItem('role',     res.role);       // "User" or "Admin"
  }
})
```

12.2 Route Guards

Angular's `CanActivate` interface lets us intercept navigation **before a component loads**. Two guards are implemented as separate injectable classes in the same file. **`AuthGuard`** is the general-purpose guard — it checks simply whether a token exists in storage, confirming the user is authenticated. **`AdminGuard`** is the role-specific guard — it checks whether the stored role is exactly `"Admin"`. If either check fails, the guard redirects the user to an appropriate page. This means even if a regular user manually types `/admin-dashboard` in the browser address bar, they will be immediately redirected — the protection is enforced at the router level, not just in the UI.

```
// core/guards/auth.guard.ts

// AuthGuard: prevents unauthenticated access to protected pages
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}
  canActivate(): boolean {
    const token = localStorage.getItem('token');
    if (token) return true;
    this.router.navigate(['/login']); // Redirect if no token
    return false;
  }
}

// AdminGuard: prevents non-admins from accessing admin panel
@Injectable({ providedIn: 'root' })
export class AdminGuard implements CanActivate {
  constructor(private router: Router) {}
  canActivate(): boolean {
    const role = localStorage.getItem('role');
    if (role === 'Admin') return true;
    this.router.navigate(['/dashboard/feed']); // Redirect non-admins
    return false;
  }
}
```

13. Angular–Backend Integration & JWT Security

13.1 JWT Middleware Configuration (Backend)

The backend is configured to **validate JWTs on every incoming request** via ASP.NET Core's built-in JWT Bearer middleware registered in `Program.cs`. The `TokenValidationParameters` object defines what constitutes a valid token: it must carry the correct issuer (`ProfileBook`), the correct audience (`ProfileBookUsers`), must not be expired (lifetime validated), and must be signed using the application's secret key. The `RoleClaimType` setting tells ASP.NET which field inside the JWT payload maps to the user's role — this is what the `[Authorize(Roles = "Admin")]` attribute reads when allowing or blocking a request. `app.UseAuthentication()` and `app.UseAuthorization()` must be called in order to activate this middleware in the request pipeline.

```
// Program.cs
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
  .AddJwtBearer(options => {
    options.TokenValidationParameters = new TokenValidationParameters {
      ValidateIssuer           = true,
      ValidateAudience         = true,
      ValidateLifetime         = true,      // Token expires after 24 hours
      ValidateIssuerSigningKey = true,
      ValidIssuer              = "ProfileBook",
      ValidAudience            = "ProfileBookUsers",
      IssuerSigningKey         = new SymmetricSecurityKey(
          Encoding.UTF8.GetBytes(builder.Configuration["JwtSettings:Token"]!)
      ),
      RoleClaimType = System.Security.Claims.ClaimTypes.Role
    };
  });
app.UseAuthentication();
app.UseAuthorization();
```

13.2 Sending Token from Angular Services

```
// core/services/user.ts
private getHeaders(): HttpHeaders {
  const token = localStorage.getItem('token');
```

```
  return new HttpHeaders({ 'Authorization': `Bearer ${token}` });
}

getAllUsers(): Observable<any[]> {
  return this.http.get<any[]>(this.apiUrl, { headers: this.getHeaders() });
}
```

13.3 End-to-End Security Flow

Step	Action
1	User logs in → POST /api/auth/login with credentials
2	API creates JWT with UserId, Username, Role claims (24-hr expiry)
3	Angular stores token + role in localStorage
4	Every secured API call adds "Authorization: Bearer <token>" header
5	ASP.NET validates signature and role → allows or returns 401/403
6	[Authorize(Roles = "Admin")] on controllers rejects non-admin JWTs with 403

14. Initial Unit Testing

14.1 Auth Service Tests (auth.spec.ts)

Unit tests are written using Angular's built-in **Jasmine/Karma** framework. Instead of making real HTTP calls (which would require a running backend), we use `HttpClientTestingModule` — this replaces the real `HttpClient` with a controlled mock. `HttpTestingController` then lets us **intercept** expected API requests, verify their HTTP method and URL, and respond with pre-defined mock data. Each test runs in isolation: `localStorage.clear()` in `beforeEach` ensures no test bleeds state into the next. The three tests below verify: (1) that the service can be instantiated correctly, (2) that a successful login correctly stores the token and role in `localStorage` as a side-effect, and (3) that the `isLoggedIn()` utility returns `true` when a token is present in storage.

```
describe('Auth Service', () => {
  let service: Auth;
```

```
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
      providers: [Auth]
    });
    service  = TestBed.inject(Auth);
    httpMock = TestBed.inject(HttpTestingController);
    localStorage.clear();
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should store token on successful login', () => {
    const mock = { token: 'fake-jwt', username: 'admin', role: 'Admin', userId: 1 };
    service.login({ username: 'admin', password: 'pass' }).subscribe((res: any) => {
      expect(localStorage.getItem('token')).toBe('fake-jwt');
      expect(localStorage.getItem('role')).toBe('Admin');
    });
    const req = httpMock.expectOne('http://localhost:5134/api/auth/login');
    expect(req.request.method).toBe('POST');
    req.flush(mock);
  });

  it('should return true for isLoggedIn when token exists', () => {
    localStorage.setItem('token', 'exists');
    expect(service.isLoggedIn()).toBe(true);
  });
});
```

14.2 User Service Tests (user.spec.ts)

This test verifies that the `getAllUsers()` service method correctly targets the expected API endpoint using a `GET` request, and that the response data flows through the observable to the subscriber without modification. Two mock users are returned, and we assert that both the **count** (2 items) and the **exact data** match the mock — confirming the service does not transform or lose any fields during the HTTP cycle.

```
it('should fetch all users', () => {
  const mockUsers = [{ userId: 1, username: 'user1' }, { userId: 2, username:
'user2' }];
```

```
  service.getAllUsers().subscribe((users: any[]) => {
    expect(users.length).toBe(2);
    expect(users).toEqual(mockUsers);
  });
  const req = httpMock.expectOne('http://localhost:5134/api/user');
  expect(req.request.method).toBe('GET');
  req.flush(mockUsers);
});
```

14.3 Post Service Tests (post.spec.ts)

This test validates the post creation flow entirely from the Angular client's perspective, with no backend involved. It confirms that when `createPost()` is called, the service issues a `POST` request to the correct URL. The mock response includes a `status: "Pending"` field, which we explicitly assert — proving that the system correctly expects and handles the **initial Pending state** of newly created posts, which is the foundation of the entire content moderation workflow. If the backend ever changed this default status, this test would immediately catch the regression.

```
it('should create a post with Pending status', () => {
  const mockResponse = { postId: 102, content: 'New post', status: 'Pending' };
  service.createPost('New post').subscribe((res: any) => {
    expect(res.postId).toBe(102);
    expect(res.status).toBe('Pending');
  });
  const req = httpMock.expectOne('http://localhost:5134/api/post');
  expect(req.request.method).toBe('POST');
  req.flush(mockResponse);
});
```